22CSH01 / 22ITH01 / 22CBH01 / 22CTH01 / 22ISH25- EXPLORATORY DATA ANALYSIS

UNIT I: EXPLORATORY DATA ANALYSIS

Topic IV: Introduction to Python -data analysis libraries

PYTHON LIBRARIES

What is A Library?

A library is a collection of utility methods, classes, and modules that your application code can use to perform specific tasks without writing the functionalities from scratch. Because libraries often have a narrow scope (for instance, Strings, Input / Output, and Sockets), their APIs (Application Programming Interfaces) are smaller and require fewer dependencies. It's nothing more than a list of class definitions. In the life of a programmer, libraries play the role of an angel as they prevent programmers from reinventing the wheel again and again and actually focus on the real problem.

What Are Python Libraries?



It is a foregone conclusion that you have probably heard of 'Python.' Python, the brainchild of Guido Van Rossum and dating back to the 1980s, has proven to be a game-changer. It is one of the most extensively used coding languages today, and it is utilized for a wide range of applications. Python's popularity can be attributed to a number of factors:

Python comes with a plethora of libraries

- Python is a beginner's programming language due to its ease and simplicity.
- Python wants their developers to be more productive in all aspects of development, deployment, and maintenance.
- Another reason for Python's enormous popularity is its portability.

- When compared to C, Java, and C++, Python's programming syntax isstraightforward to learn and has a high level of abstraction.

List of Libraries in Python

Numpy

NumPy, short for Numerical Python, is a powerful Python library for scientific computing and data analysis. It is extensively used in Exploratory Data Analysis (EDA) due to its array-oriented computing capabilities and efficient mathematical functions.



Here are some ways NumPy can be utilized in EDA:

- **Handling Data Structures**: NumPy provides the ndarray (N-dimensional array) object, which is the fundamental data structure for numerical computations. It allows efficient storage and manipulation of homogeneous data, such as numerical data in EDA. NumPy arrays provide fast element-wise operations, slicing, indexing, and reshaping capabilities, making it easier to process and analyze data.
- **Data Manipulation**: NumPy offers a wide range of functions for manipulating data. You can perform mathematical operations on arrays, such as element-wise arithmetic, logarithmic, or trigonometric operations. NumPy also provides functions for sorting, filtering, joining, and splitting arrays, enabling data restructuring and transformation during EDA.
- **Statistical Analysis**: NumPy provides various statistical functions to analyze data during EDA. You can calculate descriptive statistics like mean, median, standard deviation, variance, minimum, and maximum values using NumPy functions. NumPyalso offers correlation coefficients, percentile calculations, and histogram generationfor data exploration and summary statistics.
- **Missing Data Handling**: In EDA, dealing with missing data is crucial. NumPy provides functions to identify missing values, replace or remove them, and impute missing values using techniques like mean, median, or interpolation. These functions enable efficient handling of missing data, ensuring the integrity of the

dataanalysis process.

- **Random Number Generation**: Random sampling and generating random numbers are common in EDA for various purposes like creating synthetic datasets or simulating experiments. NumPy's random module offers functions to generate random numbers from different probability distributions, set random seeds for reproducibility, and perform random sampling from arrays.
- **Array Broadcasting**: NumPy's broadcasting feature allows for efficient and flexible computation between arrays of different shapes, eliminating the need for explicit loops. Broadcasting simplifies calculations involving arrays with different dimensions, enabling concise and efficient code for EDA tasks such as element-wise operations, vectorized calculations, or array comparisons.
- **Integration with Other Libraries**: NumPy serves as a foundation for many other data analysis and scientific computing libraries in Python. It integrates seamlessly with libraries like pandas, SciPy, matplotlib, and scikit-learn, forming a powerful ecosystem for EDA and further analysis. NumPy arrays are compatible with these libraries, allowing for smooth data interchange and analysis.

Features:

- **Interactive**: Numpy is a very interactive and user-friendly library.
- **Mathematics**: NumPy simplifies the implementation of difficult mathematical equations.
- **Intuitive**: It makes coding and understanding topics a breeze.
- **A lot of Interaction**: There is a lot of interaction in it because it is widely utilised,hence there is a lot of open source contribution.

Overall, NumPy provides essential tools and functionality for handling, manipulating, and analyzing numerical data during EDA.

Example

```
import numpy as np

arr = np.array( [[ 1, 2, 3],

                 [ 4, 2, 5]] )

print("Array is of type: ",
```

```
type(arr)) print("No. of
dimensions: ", arr.ndim)
print("Shape of array: ",
arr.shape) print("Size of array: ",
arr.size)
print("Array stores elements of type: ", arr.dtype)
```

Output:

```
Array is of type:  <class
'numpy.ndarray'>No. of dimensions:  2
Shape of array:
(2, 3)Size of
array:  6
Array stores elements of type:  int64
```

Panda

Pandas are a popular Python library for data manipulation and analysis, built on top of NumPy. It provides a flexible and efficient DataFrame data structure, which is well-suited for Exploratory Data Analysis (EDA). Pandas offer a wide range of functionalities that facilitate data exploration, preprocessing, transformation, and visualization.



Here's how Pandas can be utilized in EDA:

- **Data Loading**: Pandas provides functions to read data from various file formats such as CSV, Excel, SQL databases, and more. It allows you to import data into a DataFrame, a tabular data structure with labeled columns and rows, which is the core object for data manipulation in Pandas.
- **Data Inspection**: Pandas offers functions to quickly inspect and get a high-level overview of the data. You can use methods like head(), tail(), info(), describe(), and shape to view a sample of the data, check the data types, examine summary statistics, and get the dimensions of the DataFrame.

- **Data Cleaning**: EDA often involves dealing with missing values, duplicates, outliers, or inconsistent data. Pandas provides powerful tools to handle such data cleaning tasks. You can use functions like isnull(), dropna(), fillna(), duplicated(), and drop_duplicates() to identify and handle missing data, remove duplicates, and address data quality issues.
- **Data Manipulation**: Pandas allows you to perform a wide range of data manipulation operations during EDA. You can select specific columns or rows using indexing and slicing techniques. Pandas provides functions for filtering, sorting,grouping, aggregating, merging, and reshaping data.
- **Data Transformation**: Pandas offers methods to transform data within a DataFrame. You can apply mathematical operations, string manipulations, date/time conversions, or custom functions to columns or entire datasets. Pandas also supports the creation of new columns, data type conversions, and categorical encoding.
- **Data Visualization**: Pandas integrates well with other visualization libraries like Matplotlib and Seaborn, allowing you to create insightful visualizations during EDA. Pandas' DataFrame provides functions to generate basic plots like line plots, bar plots, scatter plots, histograms, and box plots. These visualizations help explore relationships, distributions, and patterns within the data.
- **Data Aggregation and Summary Statistics**: Pandas enables the calculation of summary statistics and aggregation of data. You can compute various statistical measures like mean, median, standard deviation, variance, correlation coefficients, and percentiles. Pandas' groupby() function allows you to group data by one ormore variables and perform aggregations on specific columns or across groups.
- **Missing Data Imputation**: Pandas provides methods to impute missing data based on different strategies such as mean, median, mode, or custom functions. These imputation techniques help handle missing values during EDA and ensure the completeness of the data for analysis.
- **Time Series Analysis**: Pandas offers specialized tools for working with time series data. It includes functionalities to handle date/time data, resample time series, perform rolling window calculations, and generate time-based visualizations. Pandas' time series capabilities are valuable in EDA when analyzing trends, seasonality, and temporal patterns.

- **Integration with Other Libraries**: Pandas seamlessly integrates with other data analysis libraries like NumPy, SciPy, scikit-learn, and more. This integration allows you to leverage the combined functionality of these libraries in EDA, modeling, and machine learning tasks. Pandas data structures can be easily converted to NumPy arrays.

Example

```
import pandas
as pdimport
numpy as npser
= pd.Series()
print(ser)
data = np.array(['g', 'e', 'e', 'k',
's'])ser = pd.Series(data)
print(ser)
```

Output:

```
Series([], dtype: float64)
0  g
1  e
2  e
3  k
4  s
dtype: object
```

Matplotlib

Matplotlib is a Python 2D plotting library that can be used for exploratory data analysis (EDA). EDA is a data analysis technique that allows you to explore your data visually to gain insights into its distribution, outliers, relationships between variables, and other characteristics.

To use Matplotlib for EDA, you first need to import the library into your Python

environment. You can do this by running the following code:

Code snippet

```
import matplotlib.pyplot as plt
```

Use code with caution. Learn more

Once you have imported Matplotlib, you can create plots by using the plt.plot() function. For example, the following code creates a histogram of the height variable in a dataset:

Code snippet

```
heights = [165, 170, 175, 180, 185]

plt.hist(hei
ghts)
plt.show()
```

This code will create a histogram with five bins, each representing a different height range. The height of each bar in the histogram will represent the number of data points that fall within that range.

You can also use Matplotlib to create other types of plots, such as box plots, scatter plots, bar charts, and pie charts. For more information on how to create these types of plots, please refer to the Matplotlib documentation.

EDA is an important data analysis technique that can be used to gain insights into your data. Matplotlib is a powerful Python library that can be used to create visualizations for EDA. By using Matplotlib, you can easily create informative and visually appealing plots that can help you to better understand your data.

Some of the most common types of visualizations that can be created with Matplotlibinclude:

- **Line plots**: Line plots are used to show the relationship between two variables. For example, you could use a line plot to show the relationship between temperature and time.
- **Bar charts**: Bar charts are used to show the distribution of data. For example, you could use a bar chart to show the distribution of heights in a population.
- **Scatter plots**: Scatter plots are used to show the relationship between two variables that are not necessarily continuous. For example, you could use a scatter plot to show

the relationship between the number of hours studied and the score on a test.

- **Histograms**: Histograms are used to show the distribution of data in a continuous variable. For example, you could use a histogram to show the distribution of heights in a population.

Here are some examples of how Matplotlib libraries can be used in Python:

- You can use Matplotlib to create line plots, bar charts, scatter plots, and histograms.
- You can use Matplotlib to customize the appearance of your visualizations.
- You can use Matplotlib to export your visualizations to a variety of file formats.
- You can use Matplotlib to embed your visualizations in web pages.

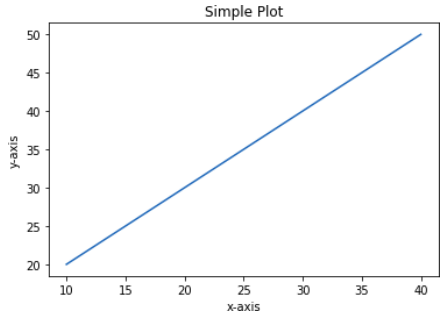
Matplotlib libraries provide a powerful and flexible platform for creating and customizing visualizations in Python.

Example

```
import matplotlib.pyplot as
pltx = [10, 20, 30, 40]
y = [20, 30, 40, 50]

plt.plot(x,          y)
plt.title("Simple
Plot")
plt.ylabel("y-
axis")
plt.xlabel("x-
axis") plt.show()
```

Output:



SciPy

SciPy is a free and open-source software library for mathematics, science, and engineering. It is also a very popular library for exploratory data analysis (EDA). EDA is a critical step in the data science process, as it allows you to understand your data and identify any potential problems.

SciPy provides a number of tools for EDA, including:

- **Descriptive statistics**: SciPy provides a number of functions for calculating descriptive statistics, such as the mean, median, and standard deviation. This information can be used to get a basic understanding of your data.
- **Data visualization**: SciPy provides a number of functions for creating visualizations, such as histograms, boxplots, and scatterplots. These visualizations can be used to identify patterns and trends in your data.
- **Hypothesis testing**: SciPy provides a number of functions for conducting hypothesis tests. This can be used to test for statistical significance in your data.

If you are new to EDA, SciPy is a great resource. It provides a number of tools that can help you to understand your data and identify any potential problems.

Here are some examples of how SciPy can be used for EDA:

Calculating descriptive statistics: You can use SciPy to calculate descriptive statistics, such as the mean, median, and standard deviation. This information can be used to get a basic understanding of your data. For example, you could use SciPy to calculate the mean age of

apopulation or the standard deviation of a set of test scores.

- **Creating visualizations:** You can use SciPy to create visualizations, such as histograms, boxplots, and scatterplots. These visualizations can be used to identify patterns and trends in your data. For example, you could use SciPy to create a histogram of the ages of a population or a boxplot of the test scores of a group of students.
- **Conducting hypothesis testing**: You can use SciPy to conduct hypothesis tests. This can be used to test for statistical significance in your data. For example, you could use SciPy to test whether the mean age of a population is different from a certain value or whether the test scores of a group of students are different from a certain value.

Features:

- SciPy's key characteristic is that it was written in NumPy, and its array makes extensive use of NumPy.
- SciPy uses its specialised submodules to provide all of the efficient numerical algorithms such as optimization, numerical integration, and many others.
- All functions in SciPy's submodules are extensively documented. SciPy's primary data structure is NumPy arrays, and it includes modules for a variety of popular scientific programming applications. SciPy handles tasks like linear algebra, integration (calculus), solving ordinary differential equations, and signal processing with ease.

Example

```
import numpy as np

A = np.array([[1,2,3],[4,5,6],[7,8,8]])

from scipy import
linalglinalg.det(A)
```

Output:

```
2.999999999999997
```

TensorFlow

TensorFlow is a high-performance numerical calculation library that is open source. It is also employed in deep learning algorithms and machine learning algorithms. It was createdby the Google Brain team researchers within the Google AI organization and is currently widely utilized by math, physics, and machine learning researchers for complicated mathematical computations.

TensorFlow is designed to be fast, and it employs techniques such as XLA (XLA or Accelerated Linear Algebra is a domain-specific compiler for linear algebra that can accelerate TensorFlow models with potentially no source code changes.) to do speedy linear algebra computations.



Features:

- **Responsive Construct**: We can easily visualize every part of the graph with TensorFlow, which is not possible with Numpy or SciKit.
- **Adaptable**: One of the most essential Tensorflow features is that it is flexible in its operation related to Machine Learning models, which means that it has modularity and allows you to make sections of it stand alone.
- **It is Simple to Train Machine Learning Models in TensorFlow**: Machine Learningmodels can be readily trained using TensorFlow on both the CPU and GPU for distributed computing.
- **Parallel Neural Network Training**: TensorFlow allows you to train many neural networks and GPUs at the same time.
- **Open Source and a large community**: Without a doubt, if it was developed by Google, there is already a significant team of software experts working on constant stability improvements. The nicest part about this machine learning library is that it is open-source, which means that anyone with internet access can use it.

There are a number of TensorFlow libraries available in Python. Some of the most popular libraries include:

- **TensorFlow Core:** This is the core TensorFlow library, and it provides the basic functionality for working with tensors, data flow graphs, and machine learning models.
- **TensorFlow Keras:** This is a high-level API for building and training deep neural networks. Keras is built on top of TensorFlow Core, and it makes it easy to create and train complex neural networks without having to worry about the underlying TensorFlow code.
- **TensorFlow Datasets:** This library provides a collection of pre-processed datasets that can be used for training and evaluating machine learning models.
- **TensorFlow Extended:** This library provides additional functionality for working with TensorFlow, such as tools for distributed training and deployment.

Here are some examples of how TensorFlow libraries can be used in Python:

- You can use the TensorFlow Core library to create a data flow graph that defines amachine learning model.
- You can use the TensorFlow Keras library to build a deep neural network for imageclassification.
- You can use the TensorFlow Datasets library to load a pre-processed dataset ofimages for training your deep neural network.
- You can use the TensorFlow Extended library to deploy your trained deep neuralnetwork to a production environment.

Example

```
import tensorflow as tf

node1 = tf.constant(3, dtype=tf.int32)

node2 = tf.constant(5, dtype=tf.int32)

node3 = tf.add(node1, node2)

sess = tf.compat.v1.Session()

print("sum of node1 and node2 is :",sess.run(node3))
sess.close()
```

Output:

Sum of node1 and node2 is: 8

Keras

Keras is a Python-based open-source neural network library that lets us experiment with deep neural networks quickly. With deep learning becoming more common, Keras emerges as a great option because, according to the creators, it is an API (Application Programming Interface) designed for humans, not machines.

Keras has a higher adoption rate in the industry and research community than TensorFlow or Theano. It is recommended that you install the TensorFlow backend engine before installing Keras.

Keras is a high-level API for building and training deep neural networks in Python. It is built on top of TensorFlow, and it makes it easy to create and train complex neural networks without having to worry about the underlying TensorFlow code.



Features:

- It runs without a hitch on both the CPU (Central Processing Unit) and GPU (Graphics Processing Unit).
- Keras supports nearly all neural network models, including fully connected, convolutional, pooling, recurrent, embedding, and so forth. These models can also be merged to create more sophisticated models.
- Keras' modular design makes it very expressive, adaptable and suited well to cutting-edge research.
- Keras is a Python-based framework, that makes it simple to debug and explore different models and projects.

Keras-powered features are already in use at various companies, for instance, Netflix, Uber, Yelp, Instacart, Zocdoc, Square, and a slew of other companies. It is particularly popular among firms that use deep learning to power their products.

Keras includes a lot of implementations of standard neural network-building elements such as layers, objectives, activation functions, optimizers, and a slew of other tools for working with picture and text data. It also includes several pre-processed data sets and pre-trained models, such as MNIST, VGG, Inception, SqueezeNet, ResNet, etc.

Keras provides a number of features that make it a popular choice for machine learning developers, including:

- A simple and intuitive API
- A wide range of pre-trained models
- Support for a variety of tasks, including image classification, natural language processing, and time series forecasting
- A well-documented and active community

Here are some examples of how Keras can be used in Python:

- You can use Keras to build a deep neural network for image classification.
- You can use Keras to build a natural language processing model for sentiment analysis.
- You can use Keras to build a time series forecasting model for predicting stock prices.
- You can use Keras to build a recommender system for suggesting products to users.Here are some of the benefits of using Keras:

- **Ease of use**: Keras has a simple and intuitive API that makes it easy to get startedwith deep learning.
- **Flexibility**: Keras is a versatile library that can be used for a wide range of tasks.
- **Performance**: Keras is a fast and efficient library that can be used to train large andcomplex models.
- **Community**: Keras has a large and active community that provides support andresources for users.

Example

```
from keras.datasets import mnist

(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

Returns:

- x_train, x_test: An unsigned integer(0-255) array of grayscale image data with shape(num_samples, 28, 28).
- y_train, y_test: An unsigned integer(0-255) array of digit labels (integers in range 0-9) with shape (num_samples,).

This dataset can be used as a drop-in replacement for MNIST. It consists of 60,000 28×28 grayscale images of 10 fashion categories, along with a test set of 10,000 images. The class labels are:

Label	Description
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

PyTorch

PyTorch is the largest machine learning library that optimizes tensor computations. It has rich APIs to perform tensor computations with strong GPU acceleration. It also helps to solve application issues related to neural networks.



Features:

- Python and its libraries are supported by PyTorch.
- Facebook's Deep Learning requirements necessitated the use of this technology.
- It provides an easy to use API that improves usability and comprehension.
- Graphs can be set up dynamically and computed dynamically at any point duringcode execution in PyTorch.
- In PyTorch, coding is simple and processing is quick.
- Because CUDA (CUDA is a parallel computing platform and application programming interface that allows software to use certain types of graphics processing unit for general purpose processing – an approach called general- purpose computing on GPUs) is supported, it can be run on GPU machines.

Example

```
import torch

t = torch.tensor([[1, 2, 3, 4],
                  [5, 6, 7, 8],
                  [9, 10, 11, 12]])

print("Reshaping")
print(t.reshape(6, 2))
print("\nResizing")
print(t.resize(2, 6))
```

```
Reshaping
tensor([[ 1,  2],
        [ 3,  4],
        [ 5,  6],
        [ 7,  8],
        [ 9, 10],
        [11, 12]])

Resizing
tensor([[ 1,  2,  3,  4,  5,  6],
        [ 7,  8,  9, 10, 11, 12]])

Transposing
tensor([[ 1,  5,  9],
        [ 2,  6, 10],
        [ 3,  7, 11],
        [ 4,  8, 12]])
```

```
print("\nTransposing")
print(t.transpose(1, 0))
```

Output:

PyBrain

The name "PyBrain" stands for Python Based Reinforcement Learning, Artificial Intelligence, and Neural Networks library. It is an open-source library built for beginners in the field of Machine Learning. It provides fast and easy-to-use algorithms for machine learning tasks. It is so flexible and easily understandable and that's why is really helpful for developers that are new in research fields.

PyGame

This library provides an easy interface to the Standard Direct media Library (SDL) platform-independent graphics, audio, and input libraries. It is used for developing video games using computer graphics and audio libraries along with Python programming language.